Using a Sledgehammer to Crack a Nut? Revisiting Automated Compiler Fault Isolation

Yibiao Yang
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
yangyibiao@nju.edu.cn

Qingyang Li
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
liqingyang@smail.nju.edu.cn

Maolin Sun
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
merlin@smail.nju.edu.cn

Jiangchang Wu
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
jiangchangwu@smail.nju.edu.cn

Yuming Zhou
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
zhouyuming@nju.edu.cn

Abstract

Background: Compilers are fundamental to software development, translating high-level source code into executable software systems. Faults in compilers can have severe consequences and thus effective localization and resolution of compiler bugs are crucial. ***Problem:*** In practice, developers often examine version history to identify and investigate bug-inducing commit (BIC) for fixing bugs. However, while numerous sophisticated Spectrum-Based Fault Localization (SBFL) techniques have been proposed for compiler fault isolation, their effectiveness has not been evaluated against the BIC-based strategies widely adopted in practice. ***Objective:*** This study aims to bridge this gap by directly comparing a BIC-based strategy, called Basic, with representative SBFL-based compiler fault isolation techniques. The BIC-based strategy closely aligns with common developer practices, as it directly identifies the BIC and treats the files modified in that commit as faulty candidates. ***Method:*** Basic strategy identifies the most recent *good release* and earliest *bad release*, and then employs a binary search to pinpoint the bug-inducing commit. All files modified in the identified commit are flagged as potentially faulty. We rigorously compare Basic against SBFL techniques using a benchmark consisting of 60 GCC bugs and 60 LLVM bugs. ***Result:*** Our analysis reveals that Basic generally outperforms state-of-the-art SBFL techniques on the critical Top-1 and Top-5 ranking metrics. ***Conclusion:*** This study provides new insights into the practical effectiveness of SBFL techniques in real-world compiler debugging scenarios. We recommend that future research adopt Basic as a baseline when developing and evaluating new compiler fault isolation methods.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3773254

CCS Concepts

• **Software and its engineering** → *Compilers*; **Software testing and debugging**.

Keywords

Compiler, Fault Isolation, Bug-inducing commit, SBFL

ACM Reference Format:
Yibiao Yang, Qingyang Li, Maolin Sun, Jiangchang Wu, and Yuming Zhou. 2026. Using a Sledgehammer to Crack a Nut? Revisiting Automated Compiler Fault Isolation. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773254

1 Introduction

Compilers are essential components in software development, responsible for translating high-level programming languages into low-level, executable machine code [6]. Due to their scale and complexity, however, compilers are prone to faults that can have severe downstream consequences [52]. Such faults often manifest as "silent" miscompilations, where incorrect binaries are produced without obvious failures, often misleading developers into attributing the observed errors to defects in their own code rather than the underlying compiler. As a result, developers may incur substantial and unnecessary debugging effort.

A substantial body of prior research has focused on testing techniques for compiler toolchains, including differential testing, metamorphic testing, and fuzzing [12, 19–21, 24, 25, 49, 52, 53, 55–57, 59]. These techniques have been highly effective in uncovering large numbers of real-world compiler bugs that require prompt resolution. Consequently, effective techniques for isolating and resolving these compiler faults are crucial for ensuring the reliability and correctness of downstream software systems [46].

Identifying the specific source files responsible for compiler bugs remains challenging due to compiler's size and complexity [9]. To address this challenge, prior studies have proposed a variety of automated Spectrum-Based Fault Localization (SBFL) techniques for locating compiler faults. These approaches construct execution

spectra by running the compiler on both failing and passing inputs, where passing inputs are typically obtained either by mutating failing programs [9, 10, 40] or by systematically adjusting compilation options [54]. Based on the collected coverage information, SBFL formulas compute suspiciousness scores for program elements, such as source files and function, to guide fault localization [2, 47].

Although SBFL techniques have demonstrated promising results for compiler fault localization [9, 10, 54], their effectiveness has rarely been examined against straightforward debugging strategies commonly employed by compiler developers in practice. In practice, developers frequently examine version histories to identify bug-inducing commits (BICs), a code change commit that introduces a bug absent in earlier versions, and analyze the associated code changes to diagnose compiler faults [29, 30]. Modern version control systems support fine-grained tracking of source code evolution. For instance, LLVM documentation explicitly recommends using `git bisect` to identify faulty revisions [30], while GCC documentation outlines procedures for identifying the release in which a bug first appears [29]. In practice, developers perform a binary search over the commit history and then inspect the files modified in the identified BIC to localize compiler faults [29, 30].

While BIC identification has been extensively studied in the context of defect prediction, bug-fix pattern mining, and automated program repair [7, 11, 18, 31, 39, 43], these efforts do not directly leverage BICs for fault localization. Among existing fault localization techniques, HSFL [42] is the only approach that explicitly incorporates BIC information. However, HSFL leverages historical BIC data solely to adjust suspiciousness scores, rather than treating the identified BIC itself as the fault localization results. More recently, Zhou et al. [61] utilized BICs to support compiler bug deduplication, further highlighting the practical value of revision history in compiler debugging. Nevertheless, despite the central role of BIC identification in real-world debugging workflows, prior studies have neither systematically examined BIC identification as a standalone, end-to-end fault localization strategy nor rigorously compared it against SBFL techniques.

In this work, we introduce Basic, a practitioner-aligned compiler fault localization approach that closely mirrors real-world debugging workflows. Specifically, Basic first determines the most recent *good* release and the earliest *bad* release, and then applies binary search to locate the BIC between them. Unlike SBFL approaches that rely on execution spectra, Basic localizes faults through release-level comparison and bisection, treating the files modified in the identified commit as faulty candidates. We conduct a comprehensive empirical study comparing Basic with representative SBFL-based compiler fault localization techniques on widely used GCC and LLVM benchmarks.

Our findings reveal the surprising effectiveness of Basic in real-world scenarios. Using benchmarks comprising 60 GCC bugs and 60 LLVM bugs, we find that Basic achieves performance comparable to, and often exceeding, that of state-of-the-art SBFL techniques, including DiWi [9], RecBi [10], LLM4CBI [40], Odfl [54], and HSFL [42], in accurately identifying faulty files. These results expose important limitations of current SBFL techniques and suggest that BIC-based localization constitutes a strong and practical baseline for compiler fault localization research.

Contributions. Our main contributions are as follows:

- **Practitioner-Centric Perspective.** We revisit compiler fault localization from the perspective of real-world debugging practice and formalize BIC identification via bisection as a standalone, end-to-end fault localization strategy for compilers.
- **Technique.** We propose Basic, a simple yet effective approach that directly identifies the BIC and treats the files modified in that commit as faulty candidates, closely align with the workflow commonly adopted by compiler developers.
- **Empirical Evaluation.** We conduct a comprehensive empirical study that systematically compares Basic with existing representative SBFL-based compiler fault localization techniques using consistent datasets and evaluation settings.
- **Implications.** Our results demonstrate that Basic performs comparable to, and in many cases outperforms, state-of-the-art SBFL-based compiler fault localization techniques. Beyond compilers, Basic is applicable to debugging a wide range of version-controlled software systems, as it relies solely on commit analysis and thus offers strong generality. These findings suggest that: (1) new fault localization techniques, especially for compilers, should demonstrate improvements not only over traditional SBFL techniques but also over practical BIC-based baselines such as Basic; and (2) hybrid strategies that integrate SBFL with BIC information may offer further benefits and warrant investigation.

2 Background & Motivation

In this section, we provide a comprehensive background on Spectrum-Based Fault Localization (SBFL) techniques used in compiler fault localization, as well as the code-change-based Version Control System (VCS) employed in modern software development. We then present the motivation behind our approach with illustrative examples of real-world bugs in GCC and LLVM compilers.

2.1 Background

2.1.1 Spectrum-Based Fault Localization. SBFL is a widely used technique in software debugging, aiming to identify the root cause of software faults or failures [4, 50]. SBFL leverages coverage information from passing and failing test cases to assess the suspiciousness of individual code elements, ranking them by their likelihood of being faulty [4]. The fundamental assumption is that program elements executed more frequently by failing test cases are more likely to be the root cause of the fault, whereas those executed more frequently by passing test cases are less likely. SBFL enables more effective debugging by prioritizing efforts on the most suspicious code locations. However, SBFL's effectiveness depends heavily on the quality and quantity of the test cases used [17, 50]. Insufficient failing test cases or inadequate representation of the fault in the test cases can compromise accuracy. Prior studies [17] have shown that increasing the number of failing test cases can enhance the effectiveness of SBFL, indicating that greater diversity in coverage information proves beneficial. The formula commonly used in SBFL to calculate the suspiciousness score of a code element is based on the this equation: $SuspiciousnessScore = Failed(e)/(Failed(e) + Passed(e))$, where $Failed(e)$ represents the number of failing test

```c
int a[6], b, c = 1, d;
short e;

void fn1 (int p) { b = a[p]; }

int main () {
  a[0] = 1;
  if (c) e--;
  d = e;
  long long f = e;
  fn1 ((f >> 56) & 1);
  printf ("%d\n", b);
  return 0;
}
```

(a) Bug-triggering test program

```
$ gcc -v
gcc version 4.9.0 (trunk 206472)
$ gcc -m64 -O1 small.c; a.out
0
$ gcc -m64 -Os small.c; a.out
1
```

(b) The compilation command that triggers the bug

```
$ svn diff -r 206417:206418 --summarize \
    | grep -E 'gcc/[A-Za-z\-]+\.c$'
M       gcc/ree.c
```

(c) The files modified in the bug-inducing revision of r206418

Figure 1: GCC bug #59747. When compiling the program with the flag 'm64 -O1' using GCC trunk revision r206472, the binary prints '0' as expected. However, when compiling with the flag 'm64 -Os', the binary erroneously prints '1'.

```c
int main() {
  int a;
  for (a = 2; a >= 0; a--)
    ;
  unsigned b = -1 % a;
  a = b;
  if (a != 0)
    __builtin_abort ();
  return 0;
}
```

(a) Bug-triggering test program

```
$ clang -v
clang version 17.0.0 (trunk ab8d7ea)
$ clang -O1 small.c; ./a.out
Aborted
```

(b) The compilation command that triggers the bug

```
$ git diff --name-only 15d5c59^ 15d5c59 -- '*.cpp'
llvm/lib/Analysis/InstructionSimplify.cpp
llvm/lib/Transforms/InstCombine/InstCombineCompares.cpp
```

(c) The files modified in the bug-inducing commit of 15d5c59

Figure 2: LLVM bug #61312 demonstrates an unexpected abortion when executing this program after compiling it using LLVM trunk revision ab8d7ea with the '-O1' compilation flag.

cases that cover code element e, and $Passed(e)$ represents the number of passing test cases that cover code element e. The suspiciousness score reflects the likelihood of a code element being faulty, with higher scores indicating higher suspicion. By ranking code elements based on their suspiciousness scores, SBFL provides guidance to developers in identifying potential fault locations for further investigation and debugging.

2.1.2 Compiler Fault Isolation Techniques. Within the context of compiler fault localization, various SBFL techniques have been developed. Chen et al. [9, 10] proposed generating witness test programs by randomly mutating bug-triggering test programs to isolate compiler optimization faults. The bug-triggering test program exposes bugs in compiler, while the witness test program is not expected to trigger the same bug with the same compilation configuration. By combining these test programs, SBFL techniques can be applied to locate faulty files in compilers [4, 38]. However, this approach has inherent limitations. There is no guarantee that the witness test program can always be generated by randomly mutating the bug-triggering test programs. The process of generating a witness test program can be time-consuming, requiring numerous mutation attempts. Moreover, the localization results obtained from witness test programs may be unreliable due to the random nature of the mutation strategies employed. These limitations make it challenging to effectively locate bugs in compilers. To address these limitations, Yang et al. [54] proposed an alternative approach focusing on searching for passing and failing compilation configurations with minimal difference. By replacing test program mutation with low-cost configuration modifications, this approach achieves more accurate and efficient fault localization. These techniques utilize coverage information from executed code to isolate and identify faults within compilers. They involve the generation of passing test programs or configurations by modifying bug-triggering test programs or utilizing fine-grained optimization options. By analyzing the coverage spectra of failing and passing compilations, these techniques calculate the suspiciousness for each program element, such as files, functions, or statements within compilers.

2.1.3 Commit-Based Version Control System. Version Control Systems (VCS) are critical role in modern software development for managing code changes. Developers use VCS to introduce modifications, or commits, which consist of specific code changes and associated message describing the changes. The commit-based approach allows for better collaboration among team members, as it enables them to work on different code changes concurrently and integrate them seamlessly. The chronological history of commits helps developers track the evolution of the codebase and promotes accountability and transparency. When a fault is reported, developers can leverage the commit history to identify the specific changes that may have introduced the issue. By pinpointing the relevant commits, developers can narrow down their focus and conduct more

targeted investigations into the code changes, leading to expedited fault isolation. In the context of fault localization, a bug-inducing commit refers to a specific commit in the VCS that is responsible for introducing a software fault. When a fault is reported, developers can leverage the commit history to identify the precise changes that may have introduced the issue. By locating the relevant bug-inducing commits, developers can narrow their focus and conduct more targeted investigations into the associated code changes. This process expedites the fault isolation process by enabling developers to effectively trace the root cause of the issue.

2.2 Motivation

Our approach is motivated by the need to evaluate and compare the performance of existing SBFL-based compiler fault isolation techniques with a simplified approach commonly employed by developers in practice. While SBFL techniques have demonstrated effectiveness in isolating compiler faults, it remains unclear whether they are practically effective when compared to the simplified bug-including commit based approach used by developers. By conducting a comprehensive comparative analysis, we aim to shed light on the effectiveness and limitations of SBFL techniques in real-world scenarios and identify areas for improvement. In the following, we illustrate our motivation using two fixed issues of the GCC and LLVM compilers.

2.2.1 Motivating Example of GCC. Figure 1 depicts the GCC bug #59747, an instance of a wrong code bug within the GCC development version r206472. Figure 1a presents the test program that triggers the bug, and Figure 1b illustrates the compilation command that exposes it. Notably, when compiling the test program using the '-m64 -O0' compilation option with GCC development version r206472, the resulting binary program produces an execution output of 0. However, when using the '-m64 -Os' option to compile the same test program, the execution output of the generated binary program yields a value of 1, an unexpected behavior. According to the bug tracking system's description page for bug #59747, developer *Jakub Jelinek* commented on the issue's initiation with r206418. Figure 1c showcases the modifications made in revision r206418, obtained through the command "`svn diff -r 206417:206418 –summarize |grep -E 'gcc/[A-Za-z\-]+\.c'`." This command displays a diff between revisions r206418 and r206417, revealing that the sole modification occurred in the 'gcc/ree.c' source file within the GCC source code induced the bug. `Jeffrey A. Law`'s comment on the description page indicates that the bug was resolved in revision r206638. Except a test program of 'pr59747.c' adding to the gcc testsuite folder, this bug-fixing revision only modified the 'gcc/ree.c' file within the gcc source code folder. The modified source file in the bug-fixing commit exactly aligns with the file modified in the bug-inducing commit. This illustrative example demonstrates the feasibility of utilizing files modified by bug-inducing commits to identify faulty files within the GCC compiler.

2.2.2 Motivating Example of LLVM. Figure 2 displays the LLVM bug #61312, reported on the LLVM GitHub issue tracker and subsequently fixed by the developers. This bug caused by incorrect code generation by the LLVM compiler, resulted in unexpected termination of the generated binary when executed. Figure 2a presents the test program that triggers this bug, while Figure 2b illustrates the compilation command that exposed the bug. After compiling the test program using the '-O1' compilation option with LLVM trunk revision ab8d7ea, the resulting binary program aborted when executed. However, the program should not terminate as the variable 'a' is set to 0, which should not activate the '`__builtin_abort()`' function. Developer `Nikita Popov` noted that this bug was introduced in revision 15d5c59. Given this information, we can utilize the command "`git diff –name-only 15d5c59^15d5c59 – 'llvm/lib/*.cpp'`" to identify the files modified in the bug-inducing commit of 15d5c59 compared to its parent commit (i.e., revision 15d5c59^). This command reveals that the bug-inducing commit modified the 'llvm/lib/Analysis/InstructionSimplify.cpp' and 'llvm/lib/Transforms/InstCombine/InstCombineCompares.cpp' source files within the LLVM source code, as indicated in Figure 2c. The inspection of the bug-fixing commit showed that the modified files were identical to those affected in the bug-inducing commit. This highlights the importance of examining bug-inducing commits to identify faulty files in the LLVM compiler.

These two examples are not isolated cases, but rather reflect a broader pattern in real-world compiler bug reports. To investigate this trend, we further conducted an empirical analysis of issue tracking systems of GCC and LLVM. For GCC, we analyzed all resolved bug reports in Bugzilla, over 40,000 in total. Among these, 6,365 reports contain the phrase "*started with*" in their discussion, which is a common way for GCC developers to indicate the bug-inducing commit [13, 14]. This suggests that these developers often use `git bisect` or similar mechanisms to identify the faulty change. Figure 3 plots the yearly number of bug reports, highlighting those that explicitly mention a bug-inducing commit. Before 2010, such references were relatively rare, but their frequency has steadily increased over time. In 2021–2022, nearly 30% of all bug reports explicitly identified a bug-inducing commit, indicating a growing reliance on this technique. This trend is likely driven by the increasing volume of bug reports, often exceeding 2,000 per year, and the efficiency gains bisection provides in fault isolation. Note that our measurement is conservative. Developers sometimes use alternative wording to indicate a bug-inducing commit, and they are not required to document the bug-inducing commit in every report, especially when the debugging process does not involve coordination with others. As a result, the actual prevalence of bisection is likely higher than what our keyword-based analysis captures. For LLVM, we performed a similar analysis using its GitHub issue tracker. From roughly 60,000 issues, we searched for indicative terms such as "*bisect*" and "*introduced in*" within issue descriptions and comments. We found 1,720 closed issues containing "*bisect*" and 1,500 containing "*introduced in*." In most of these cases, developers explicitly referenced the commit that introduced the bug. Although these keywords do not capture all bisection instances, they provide a clear signal that LLVM developers also apply bisection to isolate faults. Overall, our study shows that both GCC and LLVM developers routinely use bisection in their debugging workflows. This finding grounds our motivation for evaluating a fault localization approach that aligns with this established practice.

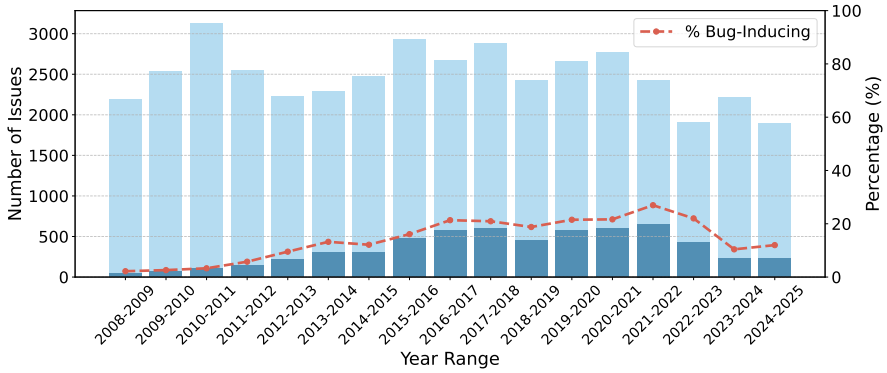


Figure 3: Bugzilla issues and BIC-associated issues over time. The blue bars represent the total number of issues, and the deeper blue bars represent the number of issues associated with BIC. The orange line indicates the percentage of BIC-associated issues relative to the total number of issues.

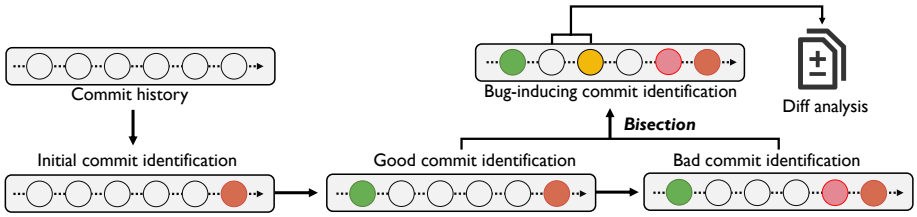


Figure 4: Overview of Basic.

3 Approach

This section introduces Basic, a compiler fault isolation technique that accurately identifies bug-inducing changes and considers all modified source code files as potential faulty files within the compiler. Basic consists of five primary steps, as shows in Figure 4. We will explain each step in detail in subsequent sections.

3.1 Framework

Algorithm 1 depicts the detailed workflow of Basic.

3.1.1 Initial Commits Identification. The first step of Basic is to retrieve the commit from the bug report description (line 1). Typically, a well-crafted bug report includes details about the specific version of the compiler where the bug manifests. This identified commit serves as the starting point for subsequent analysis, where we examine the previous commit to find the bug-inducing changes.

3.1.2 Rough Range Identification. Upon retrieving the commit from the bug report, Basic proceeds to identify the rough ranges of the *bad commit* and the *good commit*. Concretely, Basic first aims to identify the oldest major version[1] of the compiler that still manifests the bug under investigation. To accomplish this, Basic systematically examines each major version to identify the earliest one that still exhibits the bug (lines 3-6). The commit associated with this earliest failing major version is then designated as the *bad commit*. Notably, if no major version reproduces the reported bug, the initial commit is assigned as the *bad commit*. Accordingly, the *good commit* will be the commit associated with the latest passing major version of the compiler, i.e., the major version preceding the *bad commit* that does not manifest the reported bug (line 7). This preliminary investigation significantly narrows the search space for the subsequent bisection process aimed at identifying the bug-inducing commit, thus enhancing the efficiency of the fault isolation process.

3.1.3 Fine Range Identification. Subsequently, Basic identifies the fine range between the *bad commit* and the *good commit*. Basic focus on the minor versions[2] within the rough range previously identified (lines 8-14). Specifically, it first searches for the most recent minor version of the compiler that does not exhibit the reported bug. This version should be the one closest to the earliest failing major version within the rough range that does not trigger the bug. By iteratively examining the minor versions between the rough range. Basic identifies the most recent passing minor version. It then designates the commit of this version as the updated *good commit*. Next, Basic examines the minor versions within the rough range to pinpoint the earliest failing minor version. Through iterative examination, it identifies the commit of the earliest failing minor version and updates the *bad commit* accordingly. This process narrows down the search space for subsequent steps.

3.1.4 Bug-inducing Commit Identification. With the *bad commit* and *good commit* within the fine range obtained, Basic employs a binary search strategy to precisely pinpoint the specific bug-inducing commit. Binary search, a well-known algorithm, efficiently searches for a specific element in a sorted collection. In the context of Basic, the sorted collection represents the commit history between the *bad commit* and *good commit*, ordered by commit time. By iteratively

[1]A major version is defined as a significant release of the compiler, usually marked by a change in the first digit of the version number, e.g., GCC 9.1 versus GCC 10.1.

[2]Minor versions refer to subsequent releases within a major version, typically marked by a change in the second digit of the version number, e.g., GCC 9.1 versus GCC 9.2.

Algorithm 1: Compiler Faults Isolation with Basic

Input: *BR*: Bug report.
Output: *SusFiles*: Suspicious compiler files.

```
/* Initial commit retrieval                                   */
BadRev, conf, prog ← InfoRetrieve (BR)
/* Rough range identification                                 */
release_bad ← None
foreach major ∈ GetMajorReleases() do
    if Compile (major, conf, prog) == Fail then
        if ReleaseDate (major) < ReleaseDate (release_bad) then
            release_bad ← major
release_good ← GetPrecedingMajorRelease (release_bad)
/* Fine range identification                                  */
foreach minor ∈ GetMinorReleases (release_bad) do
    if Compile (minor, conf, prog) == Pass then
        if ReleaseDate (minor) > ReleaseDate (release_good) then
            release_good ← minor
    else
        if ReleaseDate (minor) < ReleaseDate (release_bad) then
            release_bad ← minor
BadRev ← GetReleaseCommit (release_bad)
GoodRev ← GetReleaseCommit (release_good)
/* Bug-inducing commit identification                         */
while BadRev − GoodRev ≠ 1 do
    rev ← Bisect (BadRev, GoodRev)
    if Compile (rev, conf, prog) == Fail then
        BadRev ← rev
    else
        GoodRev ← rev
/* Step 5: Differential Analysis                              */
SuspiciousFiles ← DiffAnalysis (BadRev, GoodRev)
return SuspiciousFiles
```

narrowing down the search space with binary search, Basic performs a series of comparisons between midpoint commits until accurately identifying the bug-inducing commit. This bisection significantly reduces the search space and accelerates *bug-inducing commit* identification.

3.1.5 Differential Analysis. Once the *bug-inducing commit* has been identified, Basic conducts a differential analysis between the bug-introducing change and its preceding change. The differential analysis aims to identify the specific code files modified by the *bug-inducing commit*. Specifically, by comparing the content and structure of the code files before and after the *bug-inducing commit*, Basic identifies the differences and determines the set of modified files. This process can be achieved by utilizing the `git diff` or `svn diff` command and the `grep` command, as exemplified in the illustrative example presented in Section 2.2. Therefore, the differential analysis enables Basic to identify the code files that were modified by the *bug-inducing commit* as the faulty files. This information is crucial for further investigation and resolution of the bug.

3.2 Illustrative Example

In this subsection, we present a concrete example to demonstrate how Basic operates. This example is derived from the debugging process of a real-world bug[3]. Initially, Basic retrieves the commit corresponding to the reported revision of the compiler that exhibits the bug. Next, Basic identifies the 'good' and 'bad' commits by examining the major and minor versions of the compiler. Once the 'good' and 'bad' commits are determined, Basic initiates the bisecting process using the command `git bisect start`, preparing the current working directory for bisecting between these commits. Basic then marks the known good and bad commits with `git bisect good <commit-hash>` and `git bisect bad <commit-hash>`, respectively. Subsequently, git automatically checks out a commit that lies about halfway between the known good and bad commits. Basic compiles the project and runs tests to determine if the bug manifests. If the midpoint commit does not exhibit the bug (i.e., the bug is absent), it is labeled as 'good' using '`git bisect good`'; otherwise, it is labeled as 'bad' using '`git bisect bad`'. This iterative process of testing and marking continues, successively halving the search range until it narrows down to zero revisions. The last commit marked as 'bad' is then confirmed as the bug-inducing commit. Finally, Basic performs a differential analysis between the bug-inducing change and its preceding change to identify the code files modified by the commit.

3.3 Implementation

We implement Basic as a practical automated compiler fault isolation tool. To achieve this, we utilize the git or svn version control system, which enables access to and extraction of the commit history of the compiler under investigation. By leveraging this resource, we trace the compiler's evolution over time, identifying specific commits that may have introduced faults. Furthermore, our implementation leverages pre-built binaries provided by developers for each release version of the compiler. Instead of recompiling the compiler for every version, we directly utilize these binaries to establish approximate ranges for identifying both the 'bad commit' and the 'good commit'. This approach offers distinct advantages in computational efficiency and significantly expedites the fault isolation process. By avoiding the need to recompile the compiler for each version, we minimize computational overhead associated with isolation. Consequently, the time required to pinpoint the faulty commit is substantially reduced, facilitating timely resolution of compiler faults. Finally, we emphasize that our implementation only reflects one realistic instantiation of the workflow commonly adopted by developers. In practice, different compiler developers may employ a variety of scripts, heuristics, and strategies when identifying bug-inducing commits [26].

4 Experimental Settings

4.1 Research Questions

In this study, we aim to address the following research questions:

- **RQ1**: How does Basic perform compared with existing compiler fault isolation techniques?
- **RQ2**: How does Basic perform relative to representative SBFL techniques under different suspiciousness formulas?
- **RQ3**: Does Basic complement state-of-the-art compiler fault isolation techniques?

[3] https://bugs.llvm.org/show_bug.cgi?id=33119

RQ1 evaluates how Basic compares with existing SBFL techniques in terms of fault localization performance. The goal is to assess whether these established methods provide essential benefits over the straightforward, developer-aligned approach used by Basic in accurately identifying faulty compiler files. *RQ2* builds on this comparison by examining whether a recent, state-of-the-art SBFL technique outperforms Basic when different suspiciousness formulas are applied. This helps us better understand the sensitivity of fault localization performance to formula choice and the relative strengths of the underlying techniques. *RQ3* explores whether Basic offers complementary benefits when used alongside SBFL techniques. Identifying such complementarity could inform future designs of hybrid approaches for compiler fault localization, offering practical guidance to researchers and practitioners.

4.2 Environment and Hardware

In order to ensure reliable and reproducible results, the experiments conducted in this study were carried out in a meticulously controlled environment. To establish a consistent and isolated experimental setting, we employed a Docker container, which offered a virtualized environment for executing the experiments. The Docker container was meticulously configured with a 20-core CPU and 120GB of memory, providing ample computational resources to effectively handle the experimental workload. Regarding the choice of the underlying operating system, we opted for Ubuntu 14.04, a 64-bit variant, as the host operating system for the Docker container. This selection was based on its compatibility with the experimental setup and the availability of necessary software dependencies. Ubuntu 14.04 is widely recognized as a well-established and extensively used Linux distribution that is highly regarded for its stability and long-term support. By employing Ubuntu 14.04, we ensured the reliability and consistency of the experimental environment. Furthermore, the utilization of Ubuntu 14.04 allows for the easy installation of various historical versions of GCC. This capability enables us to accurately determine the approximate range of bug-inducing commit, as most historical versions of GCC can be readily installed on Ubuntu 14.04.

4.3 Benchmark Selection

The selection of appropriate benchmarks plays a crucial role in comparing the effectiveness of the existing SBFL techniques with the Basic approach. Following recent work [54], we use a benchmark consisting of 60 real-world GCC bugs and 60 real-world LLVM bugs to evaluate Basic against state-of-the-art SBFL techniques. Using this benchmark enables direct and fair comparisons with prior studies. The 60 GCC bugs was originally collected by Chen et al. [9, 10] and were later used to evaluate LLM4CBI and Odfl [40, 54]. For the LLVM benchmark, 20 bugs were initially collected by Chen et al. [10], while an additional 30 bugs were collected by Yang et al. [54]. The latter Odfl study relied on the "-opt-bisect-limit" feature to determine the failing optimization passes and this feature was introduced in LLVM version 267022 (April 22, 2016).[4] To expand their evaluation, the authors re-collected 30 additional LLVM bugs, resulting in a total of 50 LLVM bugs. To maintain consistent scale of LLVM benchmark used in prior studies [9] and to further expand the evaluation set, we additionally collected 10 more LLVM bugs, yielding a total of 60 LLVM bugs in our benchmark. We select these 120 real-world compiler bugs due to the following reasons. First, all these bugs have been successfully addressed by developers, thus enabling us to identify faulty code based on bug-fixing information. Second, each bug in the benchmark was accompanied by essential information, including the lowest optimization level at which the bug was triggered, the corresponding test program that exposed the bug, the specific compiler version in which the bug was present, and the compiler files responsible for the manifestation of the bug (i.e., faulty files within compiler). This selection of benchmarks served as the ground truth against which the effectiveness of investigated techniques needed to be evaluated. By utilizing such a benchmark, we aim to provide a comprehensive and objective assessment of the capabilities and performance of existing SBFL techniques in comparison to the simplified and straightforward Basic technique. Furthermore, we chose GCC as our benchmark to enable a direct comparison between Basic and Odfl [54], as Odfl was specifically implemented for GCC. However, it is important to note that Basic is not limited to GCC but can also be applied to LLVM.

4.4 Implementation and Parameters

To ensure consistency with previous studies [9, 10, 51], we implemented and evaluated existing SBFL techniques, including RecBi, DiWi, and Odfl. To collect coverage information of the compilers, we utilized Gcov [1], specifically version 6.0.0. In cases of compatibility issues between GCC and Gcov 6.0.0, we use a compatible Gcov version. The experiments were designed deterministically to enhance overall reliability. GCC was chosen as the target compiler due to its widespread usage and large number of fine-grained optimization options, facilitating Odfl implementation.

In this study, we installed the required major and minor versions on our server. To use Basic, developers need to install necessary compiler binaries for major and minor versions on their local machines or shared servers. However, if some binaries are unavailable, they can be installed by first downloading necessary support libraries, configuring the compiler, and building and installing the commit version (e.g., install gcc version with hashid 48a320a):

```
$ git checkout 48a320a
$ ./contrib/download_prerequisites
$ mkdir buildgcc && cd buildgcc
$ ../gcc-git/configure --disable-multilib
  --disable-bootstrap --enable-languages=c,c++
  --prefix=/usr/local/48a320a
$ make -j 40 && make install
```

We developed a Python-script to automate the process of dynamically installing necessary compiler versions on demand, without requiring pre-installation. Alternatively, online resources like the Compiler-Explorer[5] platform can be used if local installation is not desired. The Compiler-Explorer maintains a comprehensive collection of compiler versions, including latest trunk versions, and provides a REST-API for programmatically interacting with the platform and accessing required compiler binaries.

[4] https://github.com/llvm/llvm-project/commit/f0f2792

[5] https://godbolt.org/

4.5 Subject SBFL techniques

We compare Basic with a set of advanced SBFL techniques, including DiWi [9], RecBi [10], LLM4CBI [40], ETEM [23], Odfl [54], and HSFL [42]. Among these, the first three are specifically designed for compiler fault isolation, while the last is a general-purpose SBFL technique. These techniques can be categorized into three groups. DiWi, RecBi, LLM4CBI, and ETEM are based on witness test programs, Odfl is based on adversarial configuration, and HSFL is based on historical spectra.

- Witness test program-based SBFL: DiWi, RecBi, LLM4CBI, and ETEM generate passing test programs by mutating on original bug-triggering test case. DiWi applies local operation mutations and uses the Metropolis-Hastings algorithm to produce diverse program variants. RecBi leverages structural mutation along with reinforcement learning to guide the mutated process. ETEM generates diverse test programs using feature mutation operators. LLM4CBI employs Large Language Model (LLM) for generating witness test programs.
- Adversarial configuration-based SBFL: Rather than mutating test programs, Odfl crafts adversarial compilation configurations by selectively adjusting fine-grained optimization options. This strategy generates both passing and failing configurations, facilitating the collection of diverse coverage data and supporting effective fault isolation.
- Historical spectrum-based SBFL: HSFL utilizes version history to aid fault localization. Its core insight is that program entities modified by bug-inducing commits, but rarely by subsequent non-inducing ones, are more likely to be faulty. HSFL first identifies bug-introducing commits, then constructs a historical spectrum-termed *Histrum*, which is orthogonal to the conventional coverage-based spectrum used in SBFL. Although HSFL was not originally designed for compiler fault isolation, it can be readily adopted to this context. We re-implemented HSFL for compilers and included it as a baseline in our evaluation of Basic.

All these SBFL techniques share the same high-level strategy: they use passing compilations to contrast against failing ones and isolate suspicious programs elements. Using both the failing and passing compilations, they compute suspiciousness scores for each program element and rank them accordingly.

To quantify the suspiciousness of each statement, a standard Spectrum-Based Fault Localization (SBFL) formula is used. Specifically, the Ochiai formula [2] is adopted, as defined in Equation 1. It considers the number of failing executions (ef_s) that cover a statement s, the number of passing executions (ep_s) that also cover s, and the number of failing executions that do not cover s (nf_s):

$$score(s) = \frac{ef_s}{\sqrt{(ef_s + nf_s)(ef_s + ep_s)}} \tag{1}$$

Notably, HSFL adjusts these suspiciousness scores using historical modification data. Specifically, for each statement s modified by a bug-inducing commit, HSFL computes a historical suspiciousness score as follow:

$$Histrum(s) = \frac{induce(s)}{\sqrt{induce(s) + noninduce(s)}} \tag{2}$$

Here, $induce(s)$ denotes the number of inducing commits that modify statement s. In compiler fault localization scenarios, we typically have a single known bug-inducing commit, so $induce(s) = 1$. The value $noninduce(s)$ represents the number of non-inducing commits that also modified s.

HSFL integrates this historical spectrum with the conventional coverage-based spectrum to produce the final suspiciousness score. For compiler fault isolation, we derive the conventional spectrum by comparing the coverage between the bug-triggering optimization level (e.g., `-O3`) and a nearby non-buggy level (e.g., `-O2`). The final HSFL score is computed as:

$$HSFL(s) = \begin{cases} (1-\alpha)\cdot SBFL(s), & \text{if } s \in \mathcal{A} \wedge s \notin S_c \\ (1-\alpha)\cdot SBFL(s) + \alpha \cdot Histrum(s), & \text{if } s \in \mathcal{A} \wedge s \in S_c \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Here, $\mathcal{A}$ is the set of statements executed during the failing execution, and S_c is the sets of statements modified in the bug-inducing commit. Following prior work [42], we set α to 0.5 by default.

Once suspicious scores are compute for each executed statement in the identified compiler source files, we aggregate them to derive a file-level suspiciousness score. In line with previous studies [9, 10, 40, 54], we compute the file score as the average of statement scores within it. This is formalized in Equation 4, where n_f is the number of statements in file f covered by failing configurations:

$$S_{avg}(f) = \sum_{i=1}^{n_f} score(s_i) \Big/ n_f \tag{4}$$

4.6 Measurements

To evaluate the effectiveness of the compiler fault localization approaches, we adopt a set of widely used evaluation metrics, consistent with prior studies [9, 10, 35, 54]. These metrics provide a comprehensive assessment of each approach's ability to accurately identify faulty components:

- **Top-N:** This metric reports the number of bugs for which at least one faulty file is ranked within the top N positions of the result list. We consider N = 1, 5, 10, and 20. A higher value indicates better performance for this metric.
- **Mean First Rank (MFR):** MFR computes the average rank of the first faulty file in the result list for each bug. It emphasizes how early the top faulty element is located. Lower values indicate better performance.
- **Mean Average Rank (MAR):** MAR measures the average rank of all faulty files for each bug, offering a broader perspective on the ranking quality. A lower MAR indicates more accurate localization across all faulty elements.

These metrics enable a thorough and multi-faceted evaluation of the techniques. Their widespread use in prior work ensures comparability and consistency with existing studies. It is important to note that Basic only considers files modified in BIC as faulty. As a result, the MFR and MAR metrics, which assume the possibility of multiple faulty files, are not applicable to Basic. Therefore, we report only the Top-N metrics when evaluating Basic.

Table 1: Compiler fault location effectiveness comparison

	Technique	Top-1		Top-5		Top-10		Top-20	
		Value	⇑	Value	⇑	Value	⇑	Value	⇑
GCC	DiWi	5.6	275.0%	19.9	75.9%	31.3	11.8%	41.7	-16.1%
	RecBi	7.7	172.7%	23.9	46.4%	34.2	2.3%	42.5	-17.7%
	LLM4CBI	9.3	125.8%	24.9	40.6%	36.4	-3.8%	44.8	-21.9%
	ETEM	20.0	5.0%	**38.0**	-7.9%	**44.0**	-20.5%	**54.0**	-35.2%
	HSFL	19.0	10.5%	22.0	59.1%	30.0	16.7%	38.0	-7.9%
	Odfl	20.0	16.7%	31.0	12.9%	42.0	-16.7%	47.0	-25.5%
	Basic	**21.0**	–	35.0	–	35.0	–	35.0	–
LLVM	ETEM	7.0	285.7%	13.0	161.5%	21.0	61.9%	28.0	21.4%
	HSFL	22.0	22.7%	23.0	47.8%	25.0	36.0%	30.0	13.3%
	Odfl	16.0	68.8%	27.0	25.9%	**36.0**	-5.6%	**47.0**	-27.7%
	Basic	**27.0**	–	**34.0**	–	34.0	–	34.0	–
ALL	ETEM	27.0	77.8%	51.0	35.3%	65.0	6.2%	82.0	-15.9%
	HSFL	41.0	17.1%	45.0	53.3%	55.0	25.5%	68.0	1.5%
	Odfl	36.0	33.3%	58.0	19.0%	**78.0**	-11.5%	**94.0**	-26.6%
	Basic	**48.0**	–	**69.0**	–	69.0	–	69.0	–

Columns "⇑" show the improvement rate of Basic over each baseline technique

5 Experimental Results

This section presents the experimental results addressing RQ1 and RQ2. Specifically, we compare Basic with existing SBFL-based compiler fault isolation techniques and assess the effectiveness of different SBFL formulae in terms of bug localization.

5.1 *RQ1: How does Basic perform compared with existing compiler fault isolation techniques?*

To ensure a fair and conservative comparison that aligns with real-world developer practices, we evaluate the performance of Basic under a worst-case setting. Specifically, consider a bug-inducing commit that modifies multiple files, denoted as $[f_1, f_2, f_3, f_4, f_5, f_6]$, where only file f_1 is the actual buggy file. In our evaluation, we conservatively assume that Basic ranks f_1 at the lowest position among the modified files, i.e., the six position in this example. Consequently, this case is counted as a failure for metrics that consider only the top-ranked files (e.g., Top-1 and Top-5). This evaluation strategy penalizes Basic as much as possible and thus avoid overestimating the effectiveness of the BIC-based approach. Consequently, the reported results represent a lower bound on the practical performance of Basic.

Furthermore, our primary objective in this section is to re-examine and compare the effectiveness of Basic with SBFL techniques in locating compiler faults at the file level. Therefore, to avoid unnecessary BIC identification, in the experimental comparison, when developers have already identified and explicitly reported the corresponding BIC on the issue page (e.g., Bugzilla for GCC or GitHub for LLVM, we directly use that commit as the BIC in our evaluation.

5.1.1 Accuracy. Table 1 presents a comparative evaluation of the fault localization effectiveness of Basic against existing SBFL techniques. Rows three to nine report results on the GCC benchmark, while rows ten to thirteen present results on the LLVM benchmark, and the final four rows summarize the aggregated across both benchmarks. The third, fifth, seventh, and ninth columns (`Value`) indicate the absolute number of bugs localized within the Top-1, Top-5, Top-10, and Top-20 ranked files, respectively. The adjacent columns (⇑) show the relative improvement (or degradation) of Basic with respect to each baseline on the corresponding metric.

As shown in Table 1, Basic consistently outperforms all state-of-the-art SBFL techniques on the crucial Top-1 metric across both compiler benchmarks. To ensure a robust and fair comparison, and to avoid inconsistencies arising from differences in implementations or experimental environments, we adopt the best-performing GCC results reported in the original ETEM and Odfl studies [23, 54], since all approaches were evaluated on the same set of 60 bugs in the GCC benchmark. In particular, Odfl achieves its best performance under the DStar formula [47]. The results for DiWi, RecBi, and LLM4CBI correspond to the averages over 10 runs reported in the LLM4CBI paper [40]. We re-ran HSFL on the same set of 60 bugs in the GCC benchmark.

For the LLVM benchmark, the results of other techniques were obtained by re-running their evaluations on the same set of 60 faults. We observe that DiWi, RecBi, and LLM4CBI perform substantially worse than techniques such as Odfl and HSFL on the GCC benchmark. Moreover, prior work has shown that DiWi and RecBi performs significantly worse than Odfl on 50 out of the 60 LLVM bugs considered in this study [54]. In addition, both DiWi, RecBi, and LLM4CBI incur considerable overhead due to extensive program mutation or frequent interactions with large language models, resulting in poor cost-effectiveness. Consequently, we do not re-evaluate these techniques on the new LLVM benchmark comprising 60 bugs.

- **GCC Benchmark:** Among the 60 bugs in the GCC benchmark, Basic successfully localizes 21 faults within the Top-1 ranked file, outperforming DiWi (5.6), RecBi (7.7), LLM4CBI (9.3), HSFL (19.0), ETEM (20.0), and Odfl (20.0). This corresponds to Top-1 improvements of 275.0%, 172.7%, 125.8%, 10.5%, 5.0%, and 5.0%, respectively. Although *ETEM* and Odfl achieve higher accuracy on the Top-5, Top-10, and Top-20 metrics, Top-1 localization is widely regarded as the most critical criterion in practice, as developers typically inspect only the highest-ranked files. Given its simplicity and closer alignment with real-world debugging workflows, Basic offers a strong practical advantage despite its conservative evaluation.
- **LLVM Benchmark:** In contrast to the GCC results, Basic demonstrates superior performance across multiple metrics on the LLVM benchmark, achieving the highest effectiveness in the Top-1 (27.0) and Top-5 (34.0) metrics. This corresponds to improvements ranging from 22.7%–285.7% for Top-1, and 25.9%–161.5% for Top-5 over existing techniques. Given the practical importance of Top-1 and Top-5 localization in compiler debugging, these results further highlight the effectiveness of leveraging historical commit information over spectrum-based techniques.

Summary of RQ1. Across both the GCC and LLVM benchmarks, even under a conservative evaluation setting that penalizes bug-inducing commits involving multiple files, Basic consistently outperforms all three state-of-the-art SBFL techniques according to

Table 2: The six adapted SBFL formulae

Name	Formula
Ochiai [2]	$\frac{ef_s}{\sqrt{(ef_s+nf_s)(ef_s+ep_s)}}$
Tarantula [16]	$\frac{ef_s/totalf_s}{ef_s/totalf_s+ep_s/totalp_s}$
Ochiai2 [28]	$\frac{ef_s np_s}{\sqrt{(ef_s+ep_s)(nf_s+np_s)(ef_s+np_s)(nf_s+ep_s)}}$
Op2 [28]	$ef_s - \frac{ep_s}{(totalp_s+1)}$
Barinel [4]	$1 - \frac{ep_s}{ep_s+ef_s}$
DStar [47]	$\frac{ef_s^\star}{ep_s+nf_s}$

We used $\star = 2$, the most thoroughly explored value [33]

Table 3: Performance comparison for Basic and Odfl in terms of different formulas

Tool	Formula	GCC				LLVM			
		Top-1	**Top-5**	**Top-10**	**Top-20**	**Top-1**	**Top-5**	**Top-10**	**Top-20**
Odfl	Ochiai	18	32	41	47	16	27	36	47
	Tarantula	14	32	39	**49**	16	29	**39**	**50**
	Ochiai2	14	26	36	46	9	23	25	21
	Op2	16	27	40	46	9	22	28	36
	Barinel	18	32	38	48	13	28	35	**50**
	DStar	20	31	**42**	47	15	25	34	45
Basic	-	**21**	**35**	35	35	**27**	**34**	34	34

Top-1 and Top-5 metrics, which are the most practically relevant indicators. For the Top-10 and Top-20 metrics, Basic achieves performance comparable to these SBFL techniques. These results indicate that a simple BIC-based strategy can outperform more sophisticated SBFL-based approaches in realistic compiler debugging scenarios. Overall, the findings of RQ1 provide empirical evidence that Basic constitutes a practical, effective, and developer-aligned baseline for compiler fault localization. This suggests that the anticipated progress in SBFL-based compiler fault isolation techniques may not have been achieved as initially envisioned. We therefore recommend adopting Basic as a strong reference approach for future research on compiler fault localization.

5.2 *RQ2: How does Basic perform relative to representative SBFL techniques under different suspiciousness formulas?*

Over the past two decades, Spectrum-Based Fault Localization (SBFL) has made significant progress, resulting in the development of a wide range of formulas for computing suspiciousness scores [48, 50]. In Section 5.1, we a adopt specific suspiciousness formula to compute statement-level scores for SBFL-based compiler fault localization techniques. However, it remains unclear whether Basic can achieve comparable or even superior performance relative to existing SBFL techniques when different suspiciousness formulas are employed.

To address this question and enable a comprehensive evaluation of Basic, we systematically compare Basic with Odfl across a diverse set of SBFL formulas. We focus exclusively on Odfl for two main reasons. First, as demonstrated in RQ1, Odfl emerges as the most cost-effective SBFL-based compiler fault localization technique: it incurs relatively low computational overhead while achieving competitive performance. Second, Odfl represents the most effective compiler fault localization approach that relies solely on SBFL, without incorporating any commit-based analysis. This makes Odfl an ideal representative for reassessing the effectiveness of pure SBFL techniques by contrasting them with Basic, a simple strategy that leverages commit-based information and aligns closely with real-world developer practices.

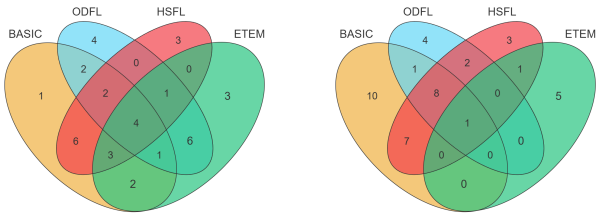
For this research question, we adopt a diverse SBFL formulae variations described in a previous study [48] to re-evaluate Odfl. Table 2 provides an overview of the SBFL formulas investigated in this study. In these formulas, ef_s, nf_s, and ep_s are defined in the same way as explained in Formula 1, while np_s denotes the number of passing cases that do not execute the statement s, and $totalf_s$ and $totalp_s$ denote the total number of failing cases and passing cases for statement s, respectively.

Table 3 presents the results of this comparison, showcasing the performance of Odfl under different suspiciousness formulas. Each row in Table 3 corresponds to a specific formula and presents the associated localization results. As shown in Table 3, the performance of Odfl varies substantially across different formulas, particularly with respect to the Top-1 and Top-5 metrics. On the GCC benchmark, the number of faults localized by Odfl under the Top-1 metric ranges from 14 to 20, while under the Top-5 metric it ranges from 26 to 32. Despite this variability, Basic consistently outperforms Odfl in terms of both Top-1 and Top-5 performance. Similar trends are observed on the LLVM benchmark. In summary, our results indicate that the SBFL-based compiler fault localization technique Odfl is sensitive to the choice of suspiciousness formula and exhibits notable performance instability. In contrast, Basic consistently outperforms Odfl on the two most critical metrics, Top-1 and Top-5, demonstrating stronger robustness across different SBFL formulas.

5.3 *RQ3: Does Basic complement state-of-the-art compiler fault isolation techniques?*

To assess whether Basic is not only effective in isolation but also complementary to state-of-the-art SBFL techniques, we analyze the overlap of faults correctly localized by Basic, HSFL, ETEM, and Odfl under the Top-1 metric. We select HSFL, ETEM, and Odfl as baselines due to their strong performance in earlier experiments in either GCC or LLVM. The results are illustrated in Figure 5.

On the GCC benchmark comprising 60 bugs (Figure 5a), Basic correctly localizes 21 faults at Top-1. Among these, 12 are not localized by Odfl, 6 are not localized by HSFL, and 11 are not detected by ETEM. Conversely, Odfl, HSFL, and ETEM correctly localize 11, 4, and 10 faults, respectively, that are not localized by Basic. On the LLVM benchmark comprising 60 bugs, Basic correctly localize 27 bugs at Top-1. Notably, among these, 17 faults are not localized by Odfl, 11 are not localized by HSFL, and 26 are not localized by ETEM, indicating that Basic identifies a substantial number of faults missed by existing SBFL techniques. In contrast, Odfl, HSFL, and ETEM together correctly localize only 6 faults that are not localized by Basic.



(a) GCC (b) LLVM

Figure 5: Distribution of correct localization achieved by Basic, HSFL, ETEM and Odfl in terms of Top-1.

Overall, these results demonstrate that Basic and SBFL techniques localize partially disjoint sets of faults, suggesting that Basic provides complementary strengths. Consequently, effectively integrating Basic with existing SBFL approaches has the potential to further improve overall compiler fault localization performance.

6 Discussion

In this section, we discuss the efficiency of Basic and the threats to validity of this study.

6.1 Efficiency

The efficiency of Basic is crucial for its practical viability in software debugging. Our analysis indicates that Basic requires approximately 1.5 hours on average to identify bug-inducing commits via binary search, positioning it as a competitive alternative to SBFL techniques such as DiWi and RecBi. The primary cost of Basic arises from setting up and installing various compiler versions during the binary search, which typically involves about six iterations. Each commit version requires roughly 800 seconds for compilation and installation, and utilizing a parallel build configuration of 40 jobs optimizes this process. In contrast, DiWi and RecBi take around five hours to generate the necessary witness test programs, which can be burdensome in fast-paced development environments where rapid bug identification is essential. Although Basic is more time-consuming than the Odfl approach (which averages about 0.6 hours for fine-grained optimization searches), it remains less costly than other SBFL techniques, further demonstrating its practicality.

6.2 Threat to Validity

Benchmark Selection: One potential threat arises from the possibility of flawed sample selection, as different samples may yield varying results. However, for direct comparison with three existing studies, a limited set of 60 defects in the GCC compiler and 60 defects in the LLVM compiler was selected as the sample. To ensure more robust conclusions, it is recommended to further analyze and compare the existing SBFL-based method with the Basic using a larger set of defect samples.

Quantitative Indicators: Our evaluation relies on common Top-k metrics (Top-1, Top-5, Top-10, and Top-20). While additional measures such as MAR and MFR can capture average ranking performance, they are not applicable to Basic, as it only considers files affected by a bug-introducing commit rather than the entire compiler. In practice, Top-1 and Top-5 are the most actionable to developers, so we expect this limitation to have minimal impact on the study's conclusions.

Method of Comparison: Another threat concerns the comparison strategy. SBFL-based methods may assign identical suspiciousness scores to multiple files. To ensure a fair comparison, we report their best-case ranking by placing the true faulty file first among ties. For Basic, we report the worst-case ranking by placing the faulty file last when multiple files are touched by the bug-introducing commit. This conservative setup provides a more reliable assessment of the relative performance of the SBFL-based method and Basic.

7 Related Work

Automated Fault Localization. Debugging is essential for helping developers understand and resolve software defects [22, 36, 37, 41, 58]. Within this process, automated fault localization plays a key role by narrowing down the parts of a program that are most likely to contain the root cause of a failure. Existing techniques include spectrum-based [4, 38, 42], slicing-based [5, 60], mutation-based [15, 27, 32], and model-based approaches [3, 44]. However, the structural and semantic complexity of compilers often limits the applicability and precision of these general-purpose techniques. To address these challenges, Chen et al. introduced DiWi [9] and RecBi [10], which leverage witness programs to aid in bug isolation, though both techniques still face constraints in effectiveness and efficiency. Tu et al. further extended this direction with LLM4CBI [40], which uses large language models to generate witness test programs. In parallel, Yang et al. proposed Odfl [51, 54], which derives passing and failing executions by adjusting compiler optimization configurations. Recent work by Zhou et al. has explored the use of historical bug-inducing commits for compiler bug deduplication [61], demonstrating the potential of BIC-based analysis for debugging compiler issues. In this study, we introduce Basic, a practical bisection-driven strategy tailored for compiler fault localization. Basic directly identifies the BIC and treats all modified files as potential suspects, aligning closely with developer practices. Our experiments show that Basic performs comparably to, and often surpasses, state-of-the-art spectrum-based techniques in isolating compiler faults.

Bug-inducing Commit Identification. Identifying bug-inducing commits is essential for debugging and fixing software issues [45]. Existing approaches fall broadly into three categories [7]. (1) Static methods analyze commit histories or bug reports without executing test cases. The canonical example is the SZZ algorithm and its variants [34], which trace from a bug-fixing commit to earlier commits that modified relevant elements. (2) Information-retrieval-based approaches treat the task as a document-retrieval problem by matching bug reports with commit logs [44]. (3) Dynamic methods employ execution information along with bug-exposing test inputs [7, 8], using coverage-based analysis to identify the most suspicious commits. In this work, we employ a binary-search-based strategy across the commit history to determine which revision introduces the observed bug. Despite is conceptual simplicity, bisection provides sufficiently precise BIC identification for our compiler fault isolation. Building on this foundation, Basic integrates BIC-driven bisection into a specialized fault-localization framework, enabling a systematic comparison with SBFL techniques.

8 Conclusion

In this study, we revisit existing Spectrum-Based Fault Localization (SBFL) techniques and conduct a rigorous evaluation their effectiveness in compared with Basic, a simple yet widely adopted debugging practice by developers that relies on bug-inducing commit analysis. Our results show that Basic generally outperforms state-of-the-art SBFL techniques in terms of both Top-1 and Top-5 localization accuracy. These findings caution us that the real progress achieved by SBFL techniques may be less substantial than previously envisaged. In light of these observations, beyond compiler faults isolation, we recommend that future research incorporate Basic as a baseline when evaluating new SBFL techniques. Furthermore, integrating Basic with existing SBFL techniques represents a promising direction, as these approaches tend to isolate complementary subsets of faults. The artifact is publicly available at: **https://doi.org/10.5281/zenodo.15030695**.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. We would like to express our appreciation to the authors of DiWi, RecBi, LLM4CBI, and Odfl for generously sharing their tools and data. Qingyang Li, Maolin Sun, and Yuming Zhou are the corresponding authors. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62472215), the Jiangsu Natural Science Foundation (Grant BK20231402), the Innovation Project of NJU-Huawei Joint Innovation Lab (Grant TC20230202021-2024-03).

References

[1] Accessed: 2023. GCOV. https://gcc.gnu.org/onlinedocs/gcc/Gcov.html.
[2] Rui Abreu, Peter Zoeteweij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Testing: Academic and industrial conference practice and research techniques-MUTATION (TAICPART-MUTATION 2007)*. 89–98.
[3] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. 2008. An observation-based model for fault localization. In *Proceedings of the 2008 International Workshop on Dynamic Analysis: held in conjunction with the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2008), WODA 2008, Seattle, Washington, USA, July 21, 2008*. 64–70.
[4] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. 2009. Spectrum-Based Multiple Fault Localization. In *ASE 2009, 24th IEEE/ACM International Conference on Automated Software Engineering, Auckland, New Zealand, November 16-20, 2009*. 88–99.
[5] Hiralal Agrawal, Richard A. DeMillo, and Eugene H. Spafford. 1993. Debugging with Dynamic Slicing and Backtracking. *Softw. Pract. Exp.* 23, 6 (1993), 589–616.
[6] V Aho Alfred, S Lam Monica, and D Ullman Jeffrey. 2007. *Compilers Principles, Techniques & Tools*. pearson Education.
[7] Gabin An, Jingun Hong, Naryeong Kim, and Shin Yoo. 2023. Fonte: Finding Bug Inducing Commits from Failures. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 589–601. doi:10.1109/ICSE48619.2023.00059
[8] Gabin An and Shin Yoo. 2021. Reducing the search space of bug inducing commits using failure coverage. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 1459–1462. doi:10.1145/3468264.3473129
[9] Junjie Chen, Jiaqi Han, Peiyi Sun, Lingming Zhang, Dan Hao, and Lu Zhang. 2019. Compiler bug isolation via effective witness test program generation. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*. 223–234.
[10] Junjie Chen, Haoyang Ma, and Lingming Zhang. 2020. Enhanced Compiler Bug Isolation via Memoized Search. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*. 78–89.
[11] Yiqun Chen, Stefan Winter, and Neeraj Suri. 2019. Inferring Performance Bug Patterns from Developer Commits. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. 70–81. doi:10.1109/ISSRE.2019.00017
[12] Hongyan Gao, Yibiao Yang, Maolin Sun, Jiangchang Wu, Yuming Zhou, and Baowen Xu. 2025. ClozeMaster: Fuzzing Rust Compiler by Harnessing LLMs for Infilling Masked Real Programs. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) *(ICSE '25)*. IEEE Press, 1422–1435. doi:10.1109/ICSE55347.2025.00175
[13] GCC Bugzilla. 2024. *Bug 103961 - [14 Regression] wrong code with -O1 -fstrict-flex-arrays=3 since r14-2580.* https://gcc.gnu.org/bugzilla/show_bug.cgi?id=103961#c11 Comment #11.
[14] GCC Bugzilla. 2024. *Bug 104334 - [14/15/16 Regression] wrong code at -O1 and above on x86-64-linux since r14-3006.* https://gcc.gnu.org/bugzilla/show_bug.cgi?id=104334#c12 Comment #12.
[15] Shin Hong, Byeongcheol Lee, Taehoon Kwak, Yiru Jeon, Bongsuk Ko, Yunho Kim, and Moonzoo Kim. 2015. Mutation-Based Fault Localization for Real-World Multilingual Programs (T). In *30th IEEE/ACM International Conference on Automated Software Engineering, ASE 2015, Lincoln, NE, USA, November 9-13, 2015*. 464–475.
[16] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *20th IEEE/ACM International Conference on Automated Software Engineering (ASE 2005), November 7-11, 2005, Long Beach, CA, USA*. 273–282.
[17] Fabian Keller, Lars Grunske, Simon Heiden, Antonio Filieri, André van Hoorn, and David Lo. 2017. A Critical Evaluation of Spectrum-Based Fault Localization Techniques on a Large-Scale Software System. In *2017 IEEE International Conference on Software Quality, Reliability and Security, QRS 2017, Prague, Czech Republic, July 25-29, 2017*. 114–125.
[18] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic Identification of Bug-Introducing Changes. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. 81–90. doi:10.1109/ASE.2006.23
[19] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) *(PLDI '14)*. Association for Computing Machinery, New York, NY, USA, 216–226. doi:10.1145/2594291.2594334
[20] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 66–79. doi:10.1145/3600006.3613140
[21] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting Compiler Testing by Injecting Real-World Code. *Proc. ACM Program. Lang.* 8, PLDI, Article 156 (June 2024), 23 pages. doi:10.1145/3656386
[22] Zenan Li, Xiaoxing Ma, Chang Xu, Jingwei Xu, Chun Cao, and Jian Lu. 2020. Operational calibration: debugging confidence errors for DNNs in the field. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 901–913. doi:10.1145/3368089.3409696
[23] Yujie Liu, Mingxuan Zhu, Jinhao Dong, Junzhe Yu, and Dan Hao. 2024. Compiler Bug Isolation via Enhanced Test Program Mutation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) *(ASE '24)*. Association for Computing Machinery, New York, NY, USA, 819–830. doi:10.1145/3691620.3695074
[24] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 196 (Nov. 2020), 25 pages. doi:10.1145/3428264
[25] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2023. Fuzzing Loop Optimizations in Compilers for C++ and Data-Parallel Languages. *Proc. ACM Program. Lang.* 7, PLDI, Article 181 (June 2023), 22 pages. doi:10.1145/3591295
[26] LLVM Community. 2021. *Using git bisect with LLVM.* https://discourse.llvm.org/t/using-git-bisect-with-llvm/32745 LLVM Discourse Forum.
[27] Seokhyeon Moon, Yunho Kim, Moonzoo Kim, and Shin Yoo. 2014. Ask the Mutants: Mutating Faulty Programs for Fault Localization. In *Seventh IEEE International Conference on Software Testing, Verification and Validation, ICST 2014, March 31 2014-April 4, 2014, Cleveland, Ohio, USA*. 153–162.
[28] Lee Naish, Hua Jie Lee, and Kotagiri Ramamohanarao. 2011. A model for spectra-based software diagnosis. *ACM Trans. Softw. Eng. Methodol.* 20, 3 (2011), 11:1–11:32.
[29] GCC online documentation. 2022. "How to Locate GCC Regressions". https://www.gnu.org/software/gcc/bugs/reghunt.html. [Online; accessed 07-Apirl-2024].
[30] LLM online documentation. 2024. "Bisecting LLVM code". https://llvm.org/docs/GitBisecting.html. [Online; accessed 07-Apirl-2024].
[31] Kai Pan, Sunghun Kim, and E. James Whitehead. 2009. Toward an understanding of bug fix patterns. *Empirical Softw. Engg.* 14, 3 (June 2009), 286–315. doi:10.1007/s10664-008-9077-5

[32] Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: mutation-based fault localization. *Softw. Test. Verification Reliab.* 25, 5-7 (2015), 605–628.

[33] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. 2017. Evaluating and improving fault localization. In *Proceedings of the 39th International Conference on Software Engineering, ICSE 2017, Buenos Aires, Argentina, May 20-28, 2017*. 609–620.

[34] Jacek Sliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *ACM SIGSOFT Softw. Eng. Notes* 30, 4 (2005), 1–5. doi:10.1145/1082983.1083147

[35] Jeongju Sohn and Shin Yoo. 2017. FLUCCS: using code and change metrics to improve fault localization. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, Santa Barbara, CA, USA, July 10 - 14, 2017*. 273–283.

[36] Yi Song, Xiaoyuan Xie, and Baowen Xu. 2024. When debugging encounters artificial intelligence: state of the art and open challenges. *Sci. China Inf. Sci.* 67, 4 (2024). doi:10.1007/S11432-022-3803-9

[37] Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. Validating SMT Rewriters via Rewrite Space Exploration Supported by Generative Equality Saturation. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 315 (Oct. 2025), 27 pages. doi:10.1145/3763093

[38] Chung Man Tang, W. K. Chan, Yuen-Tak Yu, and Zhenyu Zhang. 2017. Accuracy Graphs of Spectrum-Based Fault Localization Formulas. *IEEE Trans. Reliab.* 66, 2 (2017), 403–424.

[39] Lingxiao Tang, Chao Ni, Qiao Huang, and Lingfeng Bao. 2024. Enhancing Bug-Inducing Commit Identification: A Fine-Grained Semantic Analysis Approach. *IEEE Transactions on Software Engineering* 50, 11 (2024), 3037–3052. doi:10.1109/TSE.2024.3468296

[40] Haoxin Tu, Zhide Zhou, He Jiang, Imam Nur Bani Yusuf, Yuxian Li, and Lingxiao Jiang. 2024. Isolating Compiler Bugs by Generating Effective Witness Programs With Large Language Models. *IEEE Trans. Software Eng.* 50, 7 (2024), 1768–1788. doi:10.1109/TSE.2024.3397822

[41] Guancheng Wang, Yiqian Wu, Qihao Zhu, Yingfei Xiong, Xin Zhang, and Lu Zhang. 2023. A Probabilistic Delta Debugging Approach for Abstract Syntax Trees. In *34th IEEE International Symposium on Software Reliability Engineering, ISSRE 2023, Florence, Italy, October 9-12, 2023*. IEEE, 763–773. doi:10.1109/ISSRE59848.2023.00060

[42] Ming Wen, Junjie Chen, Yongqiang Tian, Rongxin Wu, Dan Hao, Shi Han, and Shing-Chi Cheung. 2021. Historical Spectrum Based Fault Localization. *IEEE Trans. Software Eng.* 47, 11 (2021), 2348–2368. doi:10.1109/TSE.2019.2948158

[43] Ming Wen, Yepang Liu, and Shing-Chi Cheung. 2020. Boosting automated program repair with bug-inducing commits. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results* (Seoul, South Korea) *(ICSE-NIER '20)*. Association for Computing Machinery, New York, NY, USA, 77–80. doi:10.1145/3377816.3381743

[44] Ming Wen, Rongxin Wu, and Shing-Chi Cheung. 2016. Locus: locating bugs from software changes. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE 2016, Singapore, September 3-7, 2016*, David Lo, Sven Apel, and Sarfraz Khurshid (Eds.). ACM, 262–273. doi:10.1145/2970276.2970359

[45] Ming Wen, Rongxin Wu, Yepang Liu, Yongqiang Tian, Xuan Xie, Shing-Chi Cheung, and Zhendong Su. 2019. Exploring and exploiting the correlations between bug-inducing and bug-fixing commits. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.). ACM, 326–337. doi:10.1145/3338906.3338962

[46] David B Whalley. 1994. Automatic isolation of compiler errors. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 16, 5 (1994), 1648–1659.

[47] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. The DStar Method for Effective Software Fault Localization. *IEEE Trans. Reliab.* 63, 1 (2014), 290–308.

[48] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Trans. Software Eng.* 42, 8 (2016), 707–740.

[49] Jiangchang Wu, Yibiao Yang, Maolin Sun, and Yuming Zhou. 2025. Unveiling compiler faults via attribute-guided compilation space exploration. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) *(USENIX ATC '25)*. USENIX Association, USA, Article 65, 17 pages.

[50] Xiaoyuan Xie, Tsong Yueh Chen, Fei-Ching Kuo, and Baowen Xu. 2013. A Theoretical Analysis of the Risk Evaluation Formulas for Spectrum-Based Fault Localization. *ACM Trans. Softw. Eng. Methodol.* 22, 4, Article 31 (oct 2013), 40 pages. doi:10.1145/2522920.2522924

[51] Jing Yang, Yibiao Yang, Maolin Sun, Ming Wen, Yuming Zhou, and Hai Jin. 2022. Isolating Compiler Optimization Faults via Differentiating Finer-grained Options. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 481–491. doi:10.1109/SANER53432.2022.00065

[52] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532

[53] Yibiao Yang, Yanyan Jiang, Zhiqiang Zuo, Yang Wang, Hao Sun, Hongmin Lu, Yuming Zhou, and Baowen Xu. 2019. Automatic Self-Validation for Code Coverage Profilers. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 79–90. doi:10.1109/ASE.2019.00018

[54] Yibiao Yang, Qingyang Li, Maolin Sun, Jing Yang, Jiangchang Wu, and Yuming Zhou. 2025. Isolating Compiler Faults Through Differentiated Compilation Configurations. *IEEE Transactions on Software Engineering* 51, 6 (2025), 1838–1853. doi:10.1109/TSE.2025.3569530

[55] Yibiao Yang, Maolin Sun, Yang Wang, Qingyang Li, Ming Wen, and Yuming Zhou. 2023. Heterogeneous Testing for Coverage Profilers Empowered with Debugging Support. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 670–681. doi:10.1145/3611643.3616340

[56] Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger Toolchain Validation via Cross-Level Debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 280–294. doi:10.1145/3669940.3707271

[57] Yibiao Yang, Yuming Zhou, Hao Sun, Zhendong Su, Zhiqiang Zuo, Lei Xu, and Baowen Xu. 2019. Hunting for Bugs in Code Coverage Tools via Randomized Differential Testing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 488–499. doi:10.1109/ICSE.2019.00061

[58] Huangzhao Zhang, Zhuo Li, Jia Li, Zhi Jin, and Ge Li. 2024. WELL: Applying bug detectors to bug localization via weakly supervised learning. *J. Softw. Evol. Process.* 36, 9 (2024). doi:10.1002/SMR.2669

[59] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal program enumeration for rigorous compiler testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) *(PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 347–361. doi:10.1145/3062341.3062379

[60] Xiangyu Zhang, Haifeng He, Neelam Gupta, and Rajiv Gupta. 2005. Experimental evaluation of using dynamic slices for fault location. In *Proceedings of the Sixth International Workshop on Automated Debugging, AADEBUG 2005, Monterey, California, USA, September 19-21, 2005*. 33–42.

[61] Xintong Zhou, Zhenyang Xu, and Chengnian Sun. 2025. On the Feasibility of Deduplicating Compiler Bugs with Bisection. arXiv preprint arXiv:2506.23281. https://arxiv.org/abs/2506.23281 Submitted June 29 2025.